

Reinforcement Learning on the Taxi Environment: Tabular Q-learning and a DQN Comparison

Asier Ugarteche

June 21, 2026

Abstract

We solve the Gymnasium **Taxi** environment with **tabular Q-learning** and study how hyperparameter choices affect performance. After a manually tuned baseline, we compare three automated tuning strategies — grid search, random search, and Hyperband (via Optuna). All four configurations ultimately solve the task with a 100% success rate, which illustrates a key point: on a small, deterministic environment like Taxi, hyperparameters influence *how fast* the agent learns more than *how well* it ends up performing. We close with a section on the **Deep Q-Network (DQN)** approach, which replaces the Q-table with a neural network, and discuss when that added machinery is warranted.

1 The Taxi environment

Taxi is a 5×5 grid world. The agent drives a taxi to a passenger, picks them up, drives to a destination, and drops them off. The environment has:

- **500 states** — 25 taxi positions $\times$ 5 passenger locations (four pickup spots plus “inside the taxi”) $\times$ 4 destinations;
- **6 actions** — South, North, East, West, Pickup, Dropoff;
- **rewards** — -1 per step, $+20$ for a correct dropoff, -10 for an illegal pickup/dropoff.

Crucially, Taxi is *deterministic*: a trained agent should deliver the passenger every single time. With only 500 discrete states, the problem is small enough that a lookup table over (state, action) pairs is a natural and sufficient representation — which is exactly what tabular Q-learning uses.

2 Tabular Q-learning (without DQN)

2.1 The algorithm

Q-learning stores one value $Q(s, a)$ per state–action pair in a 500×6 table, initialised to zero. At each step the agent takes an ϵ -greedy action and updates the table using the Bellman rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (1)$$

Exploration is annealed over training with an exponential schedule,

$$\epsilon(\text{ep}) = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) e^{-\text{decay_rate} \cdot \text{ep}}, \quad (2)$$

so the agent explores heavily early and increasingly exploits its learned policy later.

2.2 Hyperparameters

The behaviour of tabular Q-learning is governed entirely by a handful of hyperparameters. Table 1 lists each one, its symbol in Eqs. (1)–(2), its role, and the value used in the manual baseline (Part 1).

Hyperparameter	Symbol	Manual value	Role
learning_rate	α	0.7	step size of the Bellman update
gamma	γ	0.95	discount on future reward
total_episodes	—	25,000	number of training episodes
max_steps	—	200	step cap per episode
max_epsilon	$\epsilon_{\max}$	1.0	initial exploration rate
min_epsilon	$\epsilon_{\min}$	0.01	exploration floor
decay_rate	—	0.0005	speed of ϵ decay

Table 1: Q-learning hyperparameters and their manual-baseline values.

How each one matters.

- **learning_rate** (α): how strongly each new experience overwrites the current estimate. High values learn fast but can be unstable; low values are smoother but slower.
- **gamma** (γ): how much the agent values future reward. Close to 1 makes it far-sighted, which suits Taxi because the big +20 reward only arrives at the end of a successful trip.
- **min_epsilon** and **decay_rate**: together they shape the exploration-to-exploitation transition. A *low* **min_epsilon** lets the agent commit to its learned policy once it knows enough; **decay_rate** sets how quickly it gets there. These two turned out to be the most influential hyperparameters (Section 2.7).
- **total_episodes** and **max_steps**: the training budget and the per-episode horizon. 200 steps is comfortably above the ~ 13 steps an optimal policy needs.

Evaluation note. Throughout, the trained Q-table is evaluated with a small residual exploration rate $\epsilon = 0.05$ rather than pure greedy. On a fully deterministic environment a greedy policy would replay the exact same episode every time; keeping a little randomness gives a more realistic estimate of behaviour. This choice slightly lowers the measured reward (occasional random actions cost a step or two), which matters when comparing against the DQN results in Section 3.

2.3 Part 1 — Manual training

With the values in Table 1, the agent trained for 25,000 episodes. The learning curve climbs steeply in the first few thousand episodes and then flattens once a solid policy is found. The evaluated baseline:

avg reward 5.63, avg steps 13.6, success 100%.

2.4 Part 2 — Grid search

Grid search trains every combination of a fixed set of values: **learning_rate** $\in \{0.3, 0.7\}$, **gamma** $\in \{0.85, 0.95, 0.99\}$, **decay_rate** $\in \{0.0005, 0.002\}$, and **min_epsilon** $\in \{0.01, 0.05\}$ — 24 combinations in total. Each was trained for 10,000 episodes (cheaper than the full 25,000) and scored by the mean reward over the last 1,000 episodes. It is exhaustive but constrained to the predefined grid points. The best configuration scored 7.45:

learning_rate = 0.3, **gamma** = 0.85, **decay_rate** = 0.002, **min_epsilon** = 0.01.

The clear pattern was that a low `min_epsilon` (0.01) and a lower `learning_rate` (0.3) performed best.

2.5 Part 3 — Random search

Instead of a fixed grid, random search samples 30 configurations from continuous ranges: `learning_rate` $\sim U(0.1, 0.9)$, `gamma` $\sim U(0.80, 0.999)$, `decay_rate` $\sim U(0.0001, 0.005)$, `min_epsilon` $\sim U(0.001, 0.1)$. With the same number of trials it covers the space better than grid because it is not pinned to pre-determined points. The best trial scored 7.79:

`learning_rate` = 0.646, `gamma` = 0.828, `decay_rate` = 0.00108, `min_epsilon` = 0.00173.

The scatter-matrix analysis again singled out `min_epsilon`: lower values consistently correlated with higher scores.

2.6 Part 4 — Hyperband with Optuna

Grid and random search waste compute by training every configuration for the full budget, regardless of how it is doing. **Hyperband** runs the search like a tournament: it starts many configurations on a small budget, eliminates the worst, and gives more episodes only to the survivors. We implement this with **Optuna** (a TPE sampler combined with its **HyperbandPruner**) over the same continuous ranges as random search, for 40 trials. The best trial scored 7.82:

`learning_rate` = 0.849, `gamma` = 0.868, `decay_rate` = 0.0047, `min_epsilon` = 0.0011.

2.7 Part 5 — Final comparison

The best configuration from each method was retrained for the full 25,000 episodes and evaluated identically. Table 2 summarises the best configurations and their search scores; Table 3 reports the final evaluation.

Method	α	γ	decay_rate	min_epsilon	Search score
Manual	0.70	0.95	0.0005	0.0100	—
Grid search	0.30	0.85	0.0020	0.0100	7.45
Random search	0.646	0.828	0.00108	0.00173	7.79
Optuna/Hyperband	0.849	0.868	0.0047	0.0011	7.82

Table 2: Best configuration found by each method (search score = mean reward over the last 1,000 of 10,000 training episodes).

Analysis. Two observations stand out:

1. **All four methods solve Taxi** (100% success, ~ 13.6 – 14.0 steps), and their final rewards fall within roughly 0.6 of each other — inside the noise introduced by the $\epsilon = 0.05$ evaluation. On a deterministic environment like Taxi, hyperparameters affect *how fast* the agent converges far more than the final policy quality, since the optimal policy is unique and easily representable in a table.
2. **During the search phase**, the adaptive and random methods found higher-scoring configurations (Optuna 7.82, random 7.79) than grid (7.45), because they explored the continuous space rather than fixed grid points. After full retraining, however, those advantages washed

Method	Avg reward	Avg steps	Success rate
Manual	5.63	13.6	100%
Grid search	5.55	13.9	100%
Random search	4.99	13.8	100%
Optuna/Hyperband	5.27	14.0	100%

Table 3: Final evaluation after retraining each best config for 25,000 episodes (evaluation uses $\epsilon = 0.05$).

out: the manually chosen configuration was marginally ahead on the final evaluation. This suggests that the 10,000-episode search budget, and the residual- ϵ evaluation, leave the methods statistically indistinguishable on this problem.

The consistent thread across all searches is that `min_epsilon` and `decay_rate` are the most influential hyperparameters: a low exploration floor, reached at a sensible pace, lets the agent exploit its learned policy once it has explored enough.

3 Deep Q-Network (DQN) approach

The natural question is what changes if we replace the Q-table with a neural network — a **Deep Q-Network**. The algorithm is identical (the Bellman update of Eq. (1)); only the *representation* of Q changes.

3.1 From a Q-table to a function

A Q-table stores one number per (s, a) pair — a lookup. A DQN instead *computes* $Q(s, a)$ with a neural network. Because the network cannot consume a raw state ID (the integer 499 is not “larger” than 12 in any meaningful sense), each Taxi state is converted to a **one-hot vector** of length 500 before being fed to the network. This makes the DQN on Taxi close to a tabular method in disguise: with one-hot inputs the network learns essentially one independent column of weights per state, so there is little generalisation across states.

3.2 Architecture and key components

The DQN solution uses:

- a small multilayer perceptron, $500 \rightarrow 64 \rightarrow 64 \rightarrow 6$ (one-hot input, one Q-value per action);
- an **experience replay buffer** (capacity 50,000) that stores past transitions and samples random minibatches, breaking the correlation between consecutive steps;
- a **target network** — a slowly-updated copy of the policy network, synced every 250 steps — used to compute a stable Bellman target;
- the Huber (smooth- L_1) loss with gradient clipping, optimised by Adam.

These two stabilisers (replay buffer and target network) are the machinery that distinguishes DQN from plain online Q-learning with a function approximator.

3.3 Hyperparameters and results

Representative DQN settings: $\gamma = 0.99$, learning rate 10^{-3} , batch size 64, replay capacity 50,000, target-update period 250 steps, ϵ decayed from 1.0 to 0.05, trained for 500 episodes. Evaluated greedily ($\epsilon = 0$), it reaches:

avg reward 8.04, avg steps 13.0, success 100%.

3.4 Tabular vs. DQN

	Tabular Q-learning	DQN
Q representation	500×6 table	neural net (one-hot $\rightarrow$ MLP)
State input	integer ID (table index)	one-hot vector of length 500
Stabilisers	none needed	replay buffer + target network
Training episodes	25,000	500
Eval reward	~ 5.6 (at $\epsilon=0.05$)	8.04 (greedy)
Eval steps	~ 13.6	13.0
Success rate	100%	100%

Table 4: Tabular Q-learning versus DQN on Taxi.

The headline reward gap (8.04 vs. ~ 5.6) is largely an *evaluation-protocol artifact*: the DQN is evaluated greedily while the tabular agent keeps $\epsilon = 0.05$, so the tabular score is penalised by occasional random actions. Both approaches fully solve Taxi in ~ 13 steps with a 100% success rate. The real takeaway is that, for a 500-state problem, a Q-table is the simpler and more natural tool; DQN is introduced here as a *learning exercise* for how one would scale to state spaces too large to tabulate, where a function approximator becomes mandatory.

4 Conclusion

Tabular Q-learning solves Taxi reliably, and on this small, deterministic problem the choice of hyperparameters affects convergence speed more than final quality — every tuning method we tried reached a 100% success rate. The most influential knobs were `min_epsilon` and `decay_rate`, which control how decisively the agent shifts from exploration to exploitation. The DQN reaches the same behaviour through a neural network plus a replay buffer and a target network; on Taxi that is more machinery than the problem demands, but it is the exact recipe one needs when the state space grows beyond what a table can hold.